Summary

In this homework, you will learn how to implement Deep Q-Network. You will write and submit a report containing your answers and figures. For each part, please provide a description of your solution and any corresponding plots. **In addition, for each part, please address the following two tasks**:

- **Edge cases.** What edge cases may arise from the problem and how did you handle them? Please document edge cases of the problem that you considered and explain how you ensured that your solution has handled them correctly.
- **Tests for validating the correctness of your algorithm.** Beyond asking AI, how do you know that your algorithm is working correctly? Test whether your implementation is behaving as predicted. For example, you may change certain parameters or the structure of the problem and see if the output matches your expectation. You may also reduce the stochasticity of the environment or policy so that the process is close to deterministic. You may also find it useful to isolate the first 10 steps and observe how the states, values, rewards, and policy interact at each step.
- Implement at least one **plot** that is non-standard. For example (clearly you are not allowed to use this one): a video showing the age of 4 tuples in the replay buffer as iterations progress.

Please provide directions and documentation for how to execute validation and test experiments (what does each test do, how does it validate, etc). Provide quantitative and convincing evidence that you have sufficiently addressed the above questions. Do not confuse these requirements with unit tests or basic bug fixes, for example checking if the data is in the right format. Instead, please use creativity, critical thinking, and your understanding of RL/ML/AI to determine if the algorithm is appropriately tuned and correctly handling the specific underlying task. Finally, answer the following question: **Given an additional 3 months (extended period), what other validation tests and experiments would you perform?**

Submission Guidelines. Please submit on Canvas a pdf report with the following:

- Your GitHub username, repository name and **link to the public GitHub repository**;
- Instructions of how to run your code;
- Labeled and formatted plots required for each part;
- Responses to all discussion questions.

Please provide in your GitHub repository all code used for this assignment. **If your submission does not follow the provided guidelines, a significant number of points will be deducted from your score.** In particular, each submission should clearly provide the GitHub link to the source code for the assignment.

Deep Q-Network (100 points)

The purpose of this homework is to learn how to play games using Deep Q-Network. In each of the games mentioned below, the objective for the agent is to solve the game (a) or maximize the total score achieved during the game (b):

(a) **CartPole-v1**. You are familiar with this game from the last assignment. Use discount factor = 0.95 for this game. Your model should solve the environment by maintaining the pole upright for 500 time steps.

(b) **MsPacman-v5.** Take a look here to learn how the game is played. Train a neural net that uses images of the game as the states and models the action-value function. The discount factor is 0.99. This game has 9 actions.
You may pre-process all the images from the game with the following function before feeding them into the neural network.

```
mspacman_color = 210 + 164 + 74
def preprocess_observation(obs):
    img = obs[1:176:2, ::2] # crop and downsize
    img = img.sum(axis=2) # to greyscale
    img[img==mspacman_color] = 0 # Improve contrast
    img = (img // 3 - 128).astype(np.int8) # normalize from -128 to 127
    return img.reshape(88, 80, 1)
```

For MsPacman-v5, you will be graded based on the convergence and performance of your algorithm, specifically looking at whether your model shows learning/improvement over the training process. The model may take a long time to train (4000-6000 episodes). You may want to use additional advanced techniques for improving the performance of your algorithm, including Huber loss, learning rate scheduler, or ϵ scheduler.

For both questions:

(i) Plot the maximum Q-values versus the number of training episodes.
(ii) Plot the episode rewards versus the number training episodes, overlay with the moving average of last 100 episode rewards.
(iii) After you finish training, roll out 500 episodes using your trained neural network. Plot the histogram of the episode rewards, report the mean and standard deviation.

Grading

The grading breakdown will be as follows:

- Implementation: 20 points
- Plots and discussion: 20 points
- Performance: 20 points
- Edge cases and tests: 30 points
- Report formatting and readability: 10 points

Additional Notes

- You are responsible for training your network and creating the plots by the due time. The same rule applies to the entire class with no excuses allowed. The late homework policy still applies (see the use of Joker in the syllabus).
- You should use the specified version of both games (CartPole-v1 and MsPacman-v5).
- Your DQN implementation must use a neural network, a replay buffer, and the notion of the target network.
- You should use a single GPU card to run your program on deepdish.